Kubernetes and Cloud Native Associate - KCNA

Container Orchestration Networking

DNS in Kuberntes

Welcome to this comprehensive guide on DNS functionality within a Kubernetes cluster. In this article, we explore how names are assigned to various Kubernetes objects—such as services and pods—and illustrate the mechanisms through which pods communicate over DNS.

Consider a three-node Kubernetes cluster running multiple pods and services. Each node is assigned a unique name and IP address in your organization's DNS server. Here, our focus is solely on intra-cluster DNS resolution—how pods and services

within the cluster resolve names to communicate with one another. In most Kubernetes setups, a built-in DNS server is automatically deployed during cluster initialization (unless manually configured).

Key Concept

DNS resolution in Kubernetes enables seamless communication between components using service names rather than IP addresses, promoting scalability and easier management.

Example Scenario: Pods and Service Communication

Imagine a simple scenario with two pods and one service:

- A test pod with IP address: 10.44.1.5
- A web pod with IP address: 10.44.2.5

Even if these pods reside on different nodes, proper cluster networking ensures that DNS resolution works uniformly.

Service Creation and DNS Mapping

To allow the test pod to access the web pod, a service named **web-service** is created with its own IP address (e.g., 10.107.37.188). When this service is instantiated, the Kubernetes DNS server automatically generates a DNS record mapping the service name to its IP address. This DNS mapping allows any pod within the same namespace to address the service using its simple name.

For instance, if both pods are in the **default** namespace, the test pod can reach the web service by simply using:

```
curl http://web-service
```

```
# Expected output: Welcome to NGINX!
```

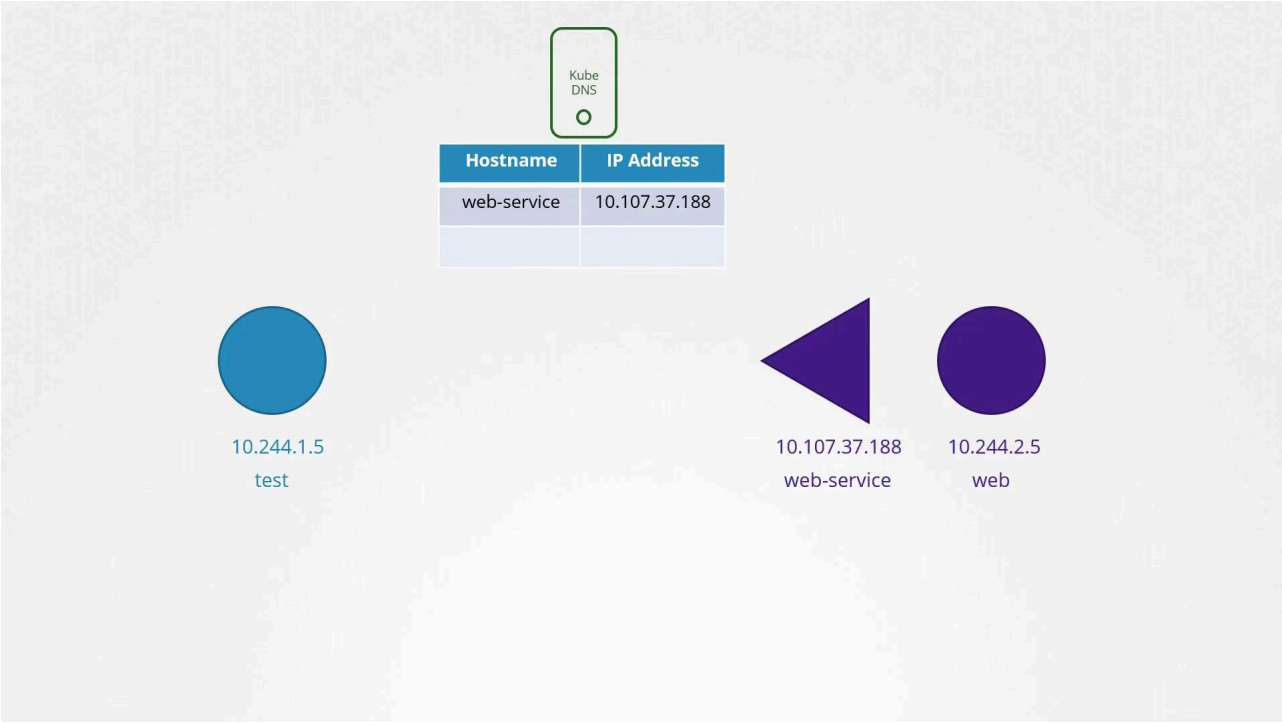
Namespace Considerations

Within a namespace, pods and services can reference each other using their short names. To access a service from a different namespace, ensure to use its fully qualified domain name.

If the web service is deployed in a different namespace, say **apps**, the test pod in the default namespace must refer to it with its fully qualified name:

```
curl http://web-service.apps
# Expected output: Welcome to NGINX!
```

In this command, **web-service** represents the service name while **apps** signifies the namespace where the service resides.



Understanding the DNS Hierarchy in Kubernetes

For every namespace, the DNS server establishes a subdomain that aggregates all of its pods and services. Moreover, all services are further organized under a subdomain named **svc**. This hierarchical structure allows you to refer to an application service using the syntax:

```
web-service.apps.svc
```

Additionally, every pod and service in the cluster belongs to a root domain, which defaults to `cluster.local`. Hence, the fully qualified domain name (FQDN) of a service appears as:

```
web-service.apps.svc.cluster.local
```

Verification within a Pod

You can verify DNS resolutions by executing the following commands from inside a pod:

```
# Access by service name within the same namespace:
curl http://web-service
# Access by service name in another namespace:
curl http://web-service.apps
# Access using the service grouped in 'svc' subdomain:
curl http://web-service.apps.svc
# Access the fully qualified domain name:
curl http://web-service.apps.svc.cluster.local
# Expected output: Welcome to NGINX!
```

DNS Records for Pods

By default, Kubernetes does not create DNS records for individual pods. However, this functionality can be enabled. When activated, Kubernetes generates DNS

records for pods based on their IP addresses, where dots are replaced with dashes. The record includes the original namespace, is categorized as a Pod type, and has `cluster.local` as the root domain.

For example, a pod with IP address 10.244.2.5 in the **default** namespace would receive a DNS record as follows:

```
curl http://10-244-2-5.apps.pod.cluster.local
# Expected output: Welcome to NGINX!
```

This naming convention ensures that both services and pods are addressed using a consistent, hierarchical format within the cluster.

Summary Table

Component	Access Method	Example Command
Pod (Default DNS)	Direct IP-based DNS record	`curl http://10-244-2-5.apps.pod.cluster.local`
Service (Same NS)	Service name only	`curl http://web-service`
Service (Cross NS)	Service name with namespace	`curl http://web-service.apps`
Service (Fully Qualified)	Full DNS hierarchy including root domain	`curl http://web-service.apps.svc.cluster.local`

Understanding the dynamic DNS resolution within a Kubernetes cluster is essential for efficient service discovery and communication. By following the hierarchical DNS naming conventions, you can ensure that all components of your cluster interact seamlessly.

For more details, check out **Kubernetes Documentation**.

Watch Video

Watch video content

Previous

CNI weave

Next

Ingress